

# EventFlow Logistics

Documentacao Tecnica - Back-end & Front-end

*Sistema de Gestao Logistica de Materiais para Eventos*

## Conteudo deste documento

1. Objetivo do sistema e dominio de negocio
2. Stack tecnico (back-end e front-end)
3. Estrutura de pastas do projeto
4. Modelo de dados (tabelas e relacionamentos)
5. Paginas e rotas do front-end
6. Endpoints REST do back-end
7. Autenticacao, autorizacao e propriedade
8. Layout, componentes e design system
9. Features importantes (regras de negocio)
10. Estado atual do layout / UX
11. Pontos fracos e dividas tecnicas
12. Recomendacoes priorizadas

*Gerado em 28/05/2026*

*Documento de referencia para o time de desenvolvimento*

## 1. Objetivo do Sistema e Dominio de Negocio

O EventFlow Logistics é um sistema web para gerenciamento de materiais de eventos. Tudo gira em torno do conceito de 'guarda-chuva do evento': cada movimentação, requisição, carregamento e devolução está vinculada a um evento específico, permitindo rastrear o ciclo de vida completo do material.

### Quem usa o sistema

- Equipe de planejamento (cria eventos e define janelas de requisição).
- Cenografia / produção (faz pedidos de materiais para o evento).
- Supervisores (aprova pedidos total, parcial ou rejeitam).
- Almoxarifado (separa material e processa movimentações).
- Logística e motoristas (planejam viagens e rotas).
- Administração (cadastros, usuários, permissões).

### Ciclo de vida típico de um evento

1. Evento é criado com datas de setup, evento e desmontagem.
2. Equipes solicitam materiais via requisição (status: rascunho -> pendente).
3. Supervisor aprova totalmente, parcialmente ou rejeita.
4. Aprovações geram ordens de carregamento (com explosão de kits).
5. Logística monta viagens (veículo + motorista + destinos).
6. Almoxarifado executa movimentações (saída -> evento -> retorno).
7. Devolução registra perdas, danos e baixas no estoque.

## 2. Stack Técnico

### Back-end

|                        |                                                                              |
|------------------------|------------------------------------------------------------------------------|
| <b>Runtime:</b>        | Node.js (ES Modules) + TypeScript 5.6                                        |
| <b>Framework:</b>      | Express.js                                                                   |
| <b>Banco de dados:</b> | PostgreSQL via Neon Serverless                                               |
| <b>ORM:</b>            | Drizzle ORM + drizzle-kit (migrações)                                        |
| <b>Validação:</b>      | Zod (schemas derivados via drizzle-zod)                                      |
| <b>Autenticação:</b>   | Passport.js (local strategy) + bcrypt + express-session (store PG)           |
| <b>Storage:</b>        | Replit Object Storage (Google Cloud Storage) - imagens de produto, CNH, etc. |
| <b>Excel:</b>          | SheetJS (xlsx) para import e export                                          |

### Front-end

|                       |                                                         |
|-----------------------|---------------------------------------------------------|
| <b>Framework:</b>     | React 18 + TypeScript + Vite                            |
| <b>Roteamento:</b>    | Wouter (alternativa leve ao React Router)               |
| <b>Estado server:</b> | TanStack Query v5 (cache, invalidação automática)       |
| <b>Formulários:</b>   | React Hook Form + Zod (zodResolver)                     |
| <b>UI:</b>            | Radix UI + shadcn/ui (estilo 'New York') + Tailwind CSS |
| <b>Ícones:</b>        | lucide-react (ícones), react-icons/si (logos)           |
| <b>Gráficos:</b>      | Recharts (dashboard)                                    |

Fonte: Inter (Google Fonts)

### 3. Estrutura de Pastas

Projeto monorepo simples, dividido em tres areas principais:

#### client/ (front-end)

- src/pages/ - paginas (uma por rota)
- src/components/ - componentes reutilizaveis (sidebar, dialogs)
- src/components/ui/ - primitivos shadcn (Button, Card, Dialog...)
- src/hooks/ - hooks (use-auth, use-toast, use-mobile)
- src/lib/ - utilitarios (queryClient, protected-route, utils)

#### server/ (back-end)

- index.ts - bootstrap do Express
- routes.ts - todas as rotas REST (~3100 linhas - grande demais)
- storage.ts - camada de acesso a dados (Repository pattern)
- auth.ts - configuracao Passport + sessao
- ownership.ts - utilitarios de permissao (admin / dono)
- optimization-engine.ts - bin packing 3D + roteamento
- vite.ts - integracao Vite (dev)

#### shared/ (tipos compartilhados)

- schema.ts - tabelas Drizzle + schemas Zod + tipos TS (~1600 linhas)

### 4. Modelo de Dados

O schema e centrado no evento. Toda movimentacao de material e rastreavel ate o evento que a originou. As principais tabelas estao agrupadas abaixo por dominio.

#### Cadastros base

| Tabela           | Campos principais                                               | Notas                          |
|------------------|-----------------------------------------------------------------|--------------------------------|
| users            | id, username, email, passwordHash, role, approvalStatus, active | Aprovacao obrigatoria de admin |
| events           | id, code, name, dates (setup/event/teardown), requestWindow     | Define janela de requisicao    |
| products         | id, sku, name, currentStock, minimumStock, ownership, type      | Owned / rented / third_party   |
| product_variants | id, productId, sku, supplierId                                  | SKU especifico de fornecedor   |
| kits + bom_lines | id, name, formula, quantity                                     | Explosao parametrica           |
| suppliers        | id, name, document, contact                                     | Fornecedores e locadores       |
| drivers          | id, name, cnh, cnhValidity, cnhImage                            | Validacao de CNH               |
| vehicle_types    | id, name, capacity, dimensions                                  | Usado no bin packing           |
| locations        | id, name, type                                                  | Galpao, evento, manutencao     |
| product_statuses | id, name                                                        | Disponivel, em uso, defeito... |

#### Requisicoes e aprovacao

| Tabela | Campos principais | Notas |
|--------|-------------------|-------|
|--------|-------------------|-------|

|                   |                                                         |                                     |
|-------------------|---------------------------------------------------------|-------------------------------------|
| material_requests | id, eventId, requestedBy(FK users), status, submittedAt | Fluxo: draft -> pending -> approved |
| request_items     | id, requestId, productId, quantity, approvedQuantity    | Aprovacao parcial por item          |
| request_audit_log | id, requestId, action, userId, timestamp                | Trilha de auditoria                 |

## Carregamento e transporte

| Tabela              | Campos principais                         | Notas                           |
|---------------------|-------------------------------------------|---------------------------------|
| loading_orders      | id, eventId, status, createdBy            | Consolida requisicoes aprovadas |
| loading_order_items | id, orderId, productId, quantity, source  | Origem: pedido/kit              |
| trips               | id, vehicleTypeId, driverId, date, status | Pode ter multiplos destinos     |
| trip_items          | id, tripId, productId, quantity           | Carga da viagem                 |
| trip_stops          | id, tripId, eventId, sequence, eta        | Sequencia otimizada             |
| docks               | id, name, available                       | Docas de carga                  |

## Movimentacoes (almoxarifado)

| Tabela          | Campos principais                                       | Notas                                |
|-----------------|---------------------------------------------------------|--------------------------------------|
| movement_groups | id, name                                                | Categoria de movimento               |
| movement_types  | id, groupId, nature, requiresApproval                   | Configuravel:<br>inbound/outbound    |
| movements       | id, typeId, status, createdBy, sourceLocId, targetLocId | Estados:<br>created/in_progress/done |
| movement_items  | id, movementId, productId, quantity, scannedAt          | Suporte a scanner                    |

## Permissoes e notificacoes

| Tabela             | Campos principais               | Notas                                 |
|--------------------|---------------------------------|---------------------------------------|
| user_roles         | userId, role                    | RBAC simples (admin, supervisor...)   |
| permissions        | id, code, description           | Granular por modulo (em planejamento) |
| role_permissions   | roleId, permissionId            | Many-to-many                          |
| notifications      | id, userId, type, payload, read | In-app + @mentions                    |
| notification_prefs | userId, channel, enabled        | Painel de preferencias                |

## 5. Paginas e Rotas do Front-end

Roteamento via Wouter. Todas as rotas (exceto /auth) sao protegidas e validam permissoes.

### Operacional

- / - Dashboard (KPIs, alertas, eventos proximos)
- /requests - Listagem de requisicoes
- /requests/:id - Detalhes + aprovacao por item
- /loading-orders - Ordens de carregamento
- /movements - Movimentacoes do almoxarifado
- /returns - Devolucoes (perdas e danos)

## Estoque e catalogo

- /inventory - Posicao atual de estoque
- /inventory/views - Visoes salvas com filtros
- /events - CRUD de eventos
- /products - CRUD de produtos + variantes
- /kits - Kits e BOM (estruturas)
- /suppliers - Fornecedores

## Transporte e relatorios

- /transport/trips - Viagens (lista + calendario)
- /reports/stock-simulation - Simulacao de faltas por data
- /reports/stock-position-simulation - Projecao temporal de estoque

## Configuracao (admin)

- /config/users - Aprovacao e gestao de usuarios
- /config/roles - Papeis e permissoes
- /config/vehicles - Tipos de veiculo
- /config/drivers - Motoristas (CNH)
- /config/locations - Localizacoes fisicas
- /config/product-statuses - Status de produtos
- /config/movement-types - Grupos e tipos de movimento

## Autenticacao

- /auth - Login, cadastro e recuperacao de senha

## 6. Endpoints REST

Todos seguem o padrao /api/<recurso>. JSON in/out. Validacao com Zod antes de chamar storage.

### Autenticacao

- POST /api/login, POST /api/register, POST /api/logout, GET /api/user
- POST /api/forgot-password, POST /api/reset-password (token)

### Eventos

- GET /api/events, GET /:id, POST /api/events, PATCH /:id, DELETE /:id
- POST /api/events/bulk - importacao Excel

### Requisicoes (com checks de propriedade)

- GET /api/requests, GET /:id, POST /api/requests
- PATCH /:id - bloqueia mudanca direta para approved/rejected
- POST /:id/approve, POST /:id/approve-partial, POST /:id/reject
- POST /:id/duplicate, POST /:id/items, DELETE /api/request-items/:id

### Carregamento e transporte

- GET/POST/PATCH /api/loading-orders

- GET/POST /api/trips, POST /api/trips/bulk
- POST /api/optimize/loading, POST /api/optimize/route

## Movimentacoes

- GET/POST /api/movements
- PATCH /:id/status - transicoes (created -> in\_progress -> done)
- POST /:id/approve - quando o tipo exige aprovacao
- POST /:id/items - itens via scanner

## Relatorios e cadastros

- POST /api/reports/stock-simulation, /stock-position-simulation
- GET/POST /api/products, /kits, /suppliers, /drivers (DELETE admin-only)
- GET/POST /api/locations, /product-statuses, /movement-types

# 7. Autenticacao, Autorizacao e Propriedade

## Autenticacao

Passport local com bcrypt. Sessao guardada em PostgreSQL (connect-pg-simple). O cliente recebe o usuario via GET /api/user e esse estado fica em cache no TanStack Query.

## Aprovacao de usuarios

Novo usuario entra com approvalStatus='pending' e nao consegue logar. Admin precisa aprovar em /config/users. Trilha de auditoria com data e responsavel.

## Autorizacao por propriedade (Phase 1)

Funcao canEditResource(user, creatorId) -> admin OU criador do recurso. Aplicada nas rotas PATCH/DELETE de requisicoes, viagens, ordens de carregamento e movimentacoes. UI esconde botoes quando o usuario nao tem permissao (camada extra ao back-end).

## Permissoes granulares (futuro)

Tabelas permissions / role\_permissions ja existem mas a interface ainda e simples. Roadmap previsto em replit.md inclui templates, hierarquia, dependencias automaticas e auditoria completa.

# 8. Layout, Componentes e Design System

## Estrutura visual

App segue layout sidebar + main. Sidebar lateral (shadcn) com grupos: Operacao, Estoque, Catalogo, Transporte, Relatorios, Configuracao. Header fino com botao de toggle da sidebar, breadcrumb implicito, sino de notificacoes e toggle de tema (dark/light).

## Design system

**Paleta:** Azul escuro (primary), azul claro, rosa (accent), roxo  
**Fonte:** Inter (300-700)

|                       |                                                 |
|-----------------------|-------------------------------------------------|
| <b>Estilo shadcn:</b> | 'New York' (mais denso e geométrico)            |
| <b>Border radius:</b> | rounded-md (consistente)                        |
| <b>Dark mode:</b>     | Implementado via class strategy (ThemeProvider) |
| <b>Elevação:</b>      | Utility hover-elevate / active-elevate-2        |

## Componentes de UI mais usados

- Card - container padrão para seções e KPIs
- Table - listagens densas (estilo ERP)
- Dialog / Sheet - edição e detalhes sem perder contexto
- Badge - status coloridos (draft, pending, approved, rejected)
- Tabs - seções dentro da mesma página (ex: detalhes de evento)
- Form + Input + Select - formulários reativos com Zod
- Toast - feedback rápido de mutations
- Notification bell - dropdown com não-lidas

## Padrões de UX

- Densidade alta (cabe muita info na tela - estilo ERP).
- Cores semânticas para status (verde/amarelo/vermelho/cinza).
- Tecla-primeiro: tabelas com foco e atalhos.
- Dialogs em vez de pages para ações rápidas (criar/editar).
- Skeleton + isPending para feedback de carregamento.

## 9. Features Importantes (Regras de Negócio)

### Janela de requisição por evento

Cada evento define requestWindowStart / End. Submissão fora da janela é bloqueada com mensagem clara. Admin pode forçar.

### Aprovação parcial

Supervisor pode aprovar quantidades por item (approvedQuantity menor que quantity). O sistema gera um 'request\_audit\_log' a cada decisão.

### Explosão paramétrica de kits

Kit pode conter uma fórmula em bom\_lines (ex: ceil(area / 4)). Quando a ordem de carregamento é gerada, o kit é expandido nos produtos finais agrupando duplicatas e mantendo a origem (kit X, requisição Y).

### Bin packing 3D para carregamento

Algoritmo First-Fit Decreasing Height aloca itens no veículo considerando peso e volume. Retorna sequência de carregamento, ocupação e alertas de excesso.

### Roteamento por vizinho mais próximo

Para viagens com vários destinos calcula sequência otimizada, distância, duração estimada e gasto de combustível.

## **Simulacao de estoque**

Cruza requisicoes pendentes/aprovadas com estoque atual e projeta faltas por data. Resultado classifica em FALTA / CRITICO / ADEQUADO com drill-down e export Excel.

## **Variantes e equivalencias**

Produtos de fornecedores diferentes podem mapear ao mesmo SKU principal. Movimentacoes resolvem automaticamente a SKU correta.

## **Tipos de movimento configuraveis**

Admin define grupos (entrada, saida, transferencia, manutencao) e tipos especificos com regras: aprovacao obrigatoria, fornecedor, status origem/destino permitidos, locais validos.

## **Notificacoes e @mentions**

Sistema in-app com sino no header. Suporta @mentions em comentarios. Painel de preferencias por canal e tipo.

## **Importacao em lote**

Eventos, produtos e plano de transporte podem ser importados via Excel. Preview, validacao linha-a-linha e relatorio de erros.

# **10. Estado Atual do Layout / UX**

## **Pontos positivos**

- Sidebar bem organizada por grupos - facil encontrar paginas.
- Densidade boa para perfil operacional.
- Dark mode funcional em todas as paginas.
- Componentes shadcn consistentes (tipografia, espacamentos).
- Cores semanticas claras para status.
- Feedback visual em mutations (toast + estados isPending).

## **Pontos fracos identificados**

- Tela de login (/auth) ainda renderiza com sidebar visivel - quebra a primeira impressao.
- Dashboard tem placeholder 'Conflitos' sem implementacao real.
- Rota /dashboard retorna 404 (so / funciona) - confunde usuario.
- Listagens grandes (requisicoes, movimentacoes) filtram tudo no front - lento com volume.
- Algumas tabelas nao tem ordenacao por coluna nem paginacao server-side.
- Falta empty-state amigavel em varias listas (so mostra tabela vazia).
- Mobile / tablet: layout assume desktop - tabelas estouram em telas menores.
- Modais muito grandes (criacao de requisicao) - poderiam ser pagina dedicada.

# **11. Pontos Fracos e Dividas Tecnicas**

## **Back-end**



- server/routes.ts com ~3100 linhas - precisa ser dividido por dominio.
- Mais de 40 'as any' espalhados - tipagem fraca em pontos criticos.
- shared/schema.ts com ~1600 linhas - dividir por agregados.
- Recuperacao de senha gera token mas nao envia e-mail (falta SMTP).
- Upload de imagens ainda usa filesystem local (pasta uploads/) em vez de Object Storage.
- Sistema de permissoes ainda simples (RBAC por role) - sem granularidade real.

## Front-end

- Filtragem feita 100% no cliente em algumas paginas.
- Validacoes duplicadas (front e back) - bom para UX mas precisa sincronizar Zod.
- Alguns componentes muito grandes (>500 linhas) misturando logica e apresentacao.
- Falta storybook / catalogo visual - dificulta evolucao do design system.
- Testes praticamente inexistentes.

## Operacional

- Sem testes automatizados.
- Sem CI configurado (lint, typecheck, build).
- Sem monitoramento (logs estruturados, metricas, alertas).

## 12. Recomendacoes Priorizadas

### Curto prazo (1-2 sprints)

- Esconder sidebar na rota /auth e criar layout dedicado de login.
- Resolver rota /dashboard (alias para /) ou remover refs no codigo.
- Adicionar empty states ilustrados em todas as listagens.
- Configurar envio de e-mail (SendGrid via integration Replit) para reset de senha.
- Migrar uploads para Replit Object Storage.
- Implementar paginacao + ordenacao server-side nas listagens grandes.

### Medio prazo (1-2 meses)

- Quebrar server/routes.ts por dominio (events.routes.ts, requests.routes.ts, etc.).
- Quebrar shared/schema.ts por agregado de negocio.
- Eliminar 'as any' progressivamente - tipar respostas de storage.
- Implementar Phase 2 de permissoes (templates, hierarquia, dependencias).
- Adicionar testes (Vitest no front, supertest no back) para fluxos criticos.

### Longo prazo

- Responsividade real para tablet (operadores no almoxarifado).
- Storybook + tokens de design publicados.
- Observabilidade (logs estruturados, OpenTelemetry, dashboards).
- Internacionalizacao (i18n) - hoje misto PT/EN.